JOB APPLICATION · QUALITY-GATED DRAFT · V2

롯데이노베이트 AI 서비스 엔지니어 자기소개서

상위 서사·기술 메커니즘·근거 충실도를 함께 반영한 5세트

핵심 포지셔닝
AI 기능을 실제 업무 흐름에 연결하고 운영 가능한 서비스로 완성하는 엔지니어

92.2 / 100 · 5세트 평균

15개 답변 · 모두 300~700자 · 모두 90점 이상 · 기준일 2026-09-07

점수는 채용 결과를 예측하지 않는 내부 첨삭 지표입니다. 저장소의 구조와 지원자의 실제 기여는 동일하지 않으므로 제출 전 본인의 변경 범위·실행 결과·성과를 확인해야 합니다.

공고의 학력 유추 정보 제외 원칙을 반영했습니다. 결제·자산관리 문장은 collab-mono에서 직접 확인되지 않아 이번 v2의 프로젝트 근거로 섞지 않았습니다.

1. 상위수준 서사 지도

NARRATIVE ARCHITECTURE

이 자기소개서는 기술 스택을 먼저 보여주는 방식이 아니라, “어떤 문제를 어떤 경계와 흐름으로 바꾸어 서비스 가치로 연결하는가”를 중심으로 구성했습니다.

업무·사용자 요구
모호한 요구와 반복되는 불편을 서비스 문제로 정의

→

구조화·경계 정의
도메인, API 계약, 실행 표면, 책임 주체를 분리

→

실행·근거 흐름
요청·실행·trace와 별도 리포팅 책임을 연결·구분

→

검증·운영 개선
실패 지점·권한·readiness·품질 신호를 기준으로 개선

Q1 · 역량

내가 어떤 엔지니어인가

기술을 업무 흐름과 운영 기준으로 번역하는 역량을 직무 수행 방식으로 설명합니다.

Q2 · 증거

그 역량을 어떻게 증명했나

collab-mono의 규모·역할·문제·선택·검증·학습을 하나의 프로젝트 여정으로 보여줍니다.

Q3 · 가치관

회사에서 어떤 방식으로 일할까

‘불편함을 개선한다’를 관찰·합의·실험·공유의 행동으로 구체화합니다.

회사 맥락과의 연결

롯데이노베이트가 다양한 산업의 업무 문제를 ICT 플랫폼·솔루션으로 전환한다는 공식 맥락에 맞춰, 특정 고객 경험을 과장하지 않고 “요구를 서비스 계약·데이터·운영 기준으로 번역하는 사람”으로 연결했습니다.

세 문항의 관계

Q1은 선언, Q2는 사례, Q3은 반복 가능한 업무 태도입니다. 같은 기술을 되풀이하지 않고 각각 능력·행동·조직 적용의 역할을 맡겨 단순 나열을 줄였습니다.

중심 문장은 공고·회사 맥락을 종합한 편집 가설이며 회사가 그대로 제시한 문구가 아닙니다. 실제 제출 전 본인의 경험과 맞는지 확인해야 합니다.

2. 기술을 메커니즘으로 번역하기

TECHNICAL MEANING

기술명은 그 자체로 점수를 얻지 않습니다. 아래처럼 **문제 → 메커니즘 → 서비스 흐름 → 품질·운영 의미**를 설명할 때만 직무 역량의 근거가 됩니다.

대상	해결하려는 문제	원고에서 설명하는 메커니즘	품질·경계 의미
계층·포트·어댑터	업무 규칙이 전송·저장·외부 SDK 변화에 끌려가는 문제	domain → application 계약 → adapter → composition으로 변화의 범위를 분리	교체 가능성·fitness·집중 테스트; 모든 장애를 예방한다고 과장하지 않음
Chat Gateway	양방향 요청 전달과 권한·취소·상태 이벤트를 함께 다뤄야 하는 문제	사용자 접점에서 세션·이벤트를 확인하고 runtime으로 전달	auth·requestRef·cancellation·smoke; loopback을 운영 성과로 쓰지 않음
Agent Runtime	서로 다른 실행 제공자와 로컬 실행 표면을 한 흐름에서 선택해야 하는 문제	조건에 따라 하나의 driver/surface를 선택하고 결과·trace 계약으로 정규화	provider readiness·출력 계약·비용/사용량 신호; 모두 동시에 활성화했다고 쓰지 않음
MCP	외부 에이전트가 기존 서비스 기능을 중복 구현하지 않고 사용해야 하는 문제	서비스 API를 재사용하는 도구·외부 에이전트 접점과 승인 흐름	입력 검증·approval·권한; 모델이나 실행 알고리즘으로 부르지 않음
Reporting·Dapr	근거·이력 보존과 서비스 간 호출을 별도 책임으로 다뤄야 하는 문제	Reporting은 HTTP/gRPC·event journal 경계, Dapr는 bounded invocation/runtime 구성	권한·재생·readiness; Chat Gateway의 직접 동기식 연결이나 ETL로 단정하지 않음

좋은 기술 문장

"외부 실행 방식이 달라져도 업무 규칙이 흔들리지 않도록 포트와 어댑터로 경계를 나누고, provider readiness와 출력 계약을 검증 기준으로 삼았습니다."

나쁜 기술 문장

"OpenAI Agents, Strands, CLI, MCP, Dapr, gRPC를 사용해 안정적인 AI 플랫폼을 구축했습니다." — 역할·선택 조건·실행 결과가 없어 방어할 수 없습니다.

공고가 요구하는 API·데이터·플랫폼·품질 역량을 저장소의 구체적 메커니즘과 연결하되, 확인되지 않은 모델 학습·클라우드 운영·프로덕션 지표는 제외했습니다.

3. 세분화 평가 체계

100 POINTS · 12 AXES

전체 점수는 12개 축의 가중합이지만, 기술 정확성·근거 충실도·면접 방어력은 다른 점수로 상쇄하지 않는 별도 게이트로 운영했습니다.

ID	평가축	배점	90점대에서 의미하는 것
A1	**서사 일관성**	**8**	Q1 역량·Q2 증거·Q3 업무 태도가 하나의 엔지니어 정체성으로 이어짐
A2	**문항 충족**	**7**	필수 요소가 체크리스트가 아닌 이야기 속에 들어 있음
A3	**직무 적합성**	**12**	AI API·데이터 흐름·플랫폼 운영·발굴·품질과 행동이 연결됨
A4	**기업·사업 맥락**	**6**	다양한 업무 문제를 플랫폼·서비스로 전환하는 관점이 자연스러움
A5	**근거 충실도·귀속**	**12**	사실·해석·개인 기여·실행 결과·미래 계획의 시제와 출처가 분리됨
A6	**맥락·도메인 깊이**	**7**	무엇을 만들었는지뿐 아니라 어떤 업무 제약을 풀었는지 설명함
A7	**주도성·판단**	**8**	본인의 우선순위·대안·변경·검증이 후속 질문으로 방어 가능함
A8	**인과적 수행 논리**	**10**	문제 → 판단 → 구현 → 검증 → 학습의 흐름이 끊기지 않음
A9	**기술 메커니즘 정확성**	**10**	기술의 계층·입출력·선택 조건·실패 의미를 정확히 설명함
A10	**운영·품질·리스크**	**8**	auth·readiness·trace·실패·복구를 서비스 조건으로 다룸
A11	**결과·학습·성장**	**6**	배운 점이 다음 직무에서 반복 가능한 행동으로 연결됨
A12	**차별성·가독성·방어력**	**6**	짧게 읽히면서도 why/how/what failed 질문에 답할 수 있음

메커니즘 깊이 · 0–4

0 스택 나열 → **1** 문제만 제시 → **2** 한 기술의 역할 설명 → **3** 세 단계 이상의 흐름과 실패·품질 신호 → **4** 대안·운영 신호·관찰 결과 또는 범위가 명시된 미래 계획

근거·면접 방어 · 0–4

0 근거 없음 → **2** 구조·README 근거 → **3** 사실/기여/미래를 분리하고 why/how 답변 가능 → **4** 변경 파일·대안·실패·측정·제외할 주장까지 방어

90–100점은 제출 가능한 편집 초안, 85–89점은 인과·성과·중복 중 하나를 보강해야 하는 초안, 70–84점은 구조는 있으나 나열식이어서 제외, 70점 미만은 재작성으로 해석합니다.

4. 근거 경계와 제외 기준

CLAIM LEDGER

유형	의미	안전한 표현	피해야 할 표현
F	**공식 공고·저장소에서 확인한 사실**	"공고는 …을 담당 업무로 제시합니다." "저장소는 …을 정의합니다."	구조가 곧 개인 성과라고 단정하기
U	**사용자가 제공한 현재 프로젝트 맥락**	본인 확인 후 "개인 프로젝트로 진행했습니다."	저장소만으로 전체 저자·리더십 확정
P	**본인의 설계·구현·판단**	변경 파일·결정·대안·검증 방법을 함께 제시	모든 모듈을 혼자 주도했다고 자동 확장
O	**실제 실행 결과·지표·장애 기록**	날짜·명령·테스트·metric·incident가 있을 때만 과거형	CI/README 존재만으로 "모두 통과·운영 안정" 주장
I	**근거에 기반한 제한적 해석**	"변경 범위를 줄이는 데 기여하도록 설계했습니다."	"보장합니다·예방합니다·항상 안정적입니다."
N	**입사 후 실천 계획**	"정의하겠습니다·검증하겠습니다·고도화하겠습니다."	아직 하지 않은 운영 개선을 과거 성과로 쓰기

이번 결과물에서 제외한 주장

- Python/PyTorch 모델 학습·파인튜닝
- Kubernetes·AWS·Azure 실제 운영
- 사용자 수·트래픽·지연·정확도·비용 절감·가동률 수치
- Chat Gateway와 Reporting의 직접 동기식 연결
- MCP를 모델·실행 알고리즘으로 표현

제출 전 본인 확인

- 실제로 맡은 모듈·변경 파일·핵심 결정을 확인
- 실제로 실행한 테스트·배포·검증 결과가 있으면 날짜와 근거 추가
- 실패 사례와 거절한 대안 한 가지씩 준비
- 확인되지 않은 문장은 미래 계획 또는 제한적 해석으로 유지

주요 근거 링크

- 롯데그룹 공식 채용 공고 · 롯데이노베이트 계열사 소개 · 공식 인사제도
- pociie/collab-mono · root package.json · Agent Runtime README · Chat Gateway README · Reporting README

세트 1 · 흐름을 연결해 운영 가능한 AI 서비스로 완성

가장 균형적인 기본안 · 직무 적합성·프로젝트 근거·운영 관점

문항 1	문항 2	문항 3	세트
94	95	93	94 · PASS

1. 지식 및 기술역량을 지원직무에서 발휘하는 방법

518자

AI 서비스 엔지니어의 역할은 모델을 호출하는 코드를 만드는 데서 끝나지 않고, 사용자의 요청이 데이터·API·실행·검증의 흐름을 통과해 신뢰 가능한 결과로 돌아오게 만드는 것이라고 생각합니다. 제가 갖춘 역량은 복잡한 요구를 도메인 개념과 서비스 경계로 구조화하는 능력, TypeScript 기반 백엔드와 웹·외부 시스템을 연결하는 설계 능력, 그리고 테스트·상태·로그를 통해 동작을 확인하는 습관입니다. collab-mono에서는 대화·게이트웨이·실행 모듈과 리포팅 서비스의 책임을 구분하고, 요청·실행·추적 정보가 각 경계에서 어떻게 이동하는지 설계했습니다. 계층과 포트를 나눈 이유는 기술을 보여주기 위해서가 아니라 업무 규칙이 전송 방식이나 저장소 변화에 끌려가지 않게 하기 위해서였습니다. 입사 후에는 요구사항을 API와 데이터 흐름으로 번역하고, 실패 지점과 검증 기준을 먼저 정의하겠습니다. 이를 바탕으로 AI 기능을 기존 업무에 자연스럽게 연결하고, 품질 확인과 운영 개선이 이어지는 서비스로 발전시키겠습니다.

평가 포인트: 기술을 업무 흐름·경계·운영 의미로 연결 · 모델 연구 경험으로 과장하지 않음

2. 프로젝트를 완수하기 위해 노력한 경험

500자

collab-mono는 TypeScript 기반 npm workspace 모노레포로, 대화·에이전트 실행·협업·리포팅·MCP·웹·인프라·E2E 영역을 함께 다루는 개인 프로젝트입니다. 저는 기획자와 개발자, 운영 검증자의 역할을 혼자 맡아 요구를 서비스 흐름으로 풀고 구조 설계부터 구현·검증까지 주도했습니다. 범위가 넓어질수록 기능을 먼저 붙이면 모듈 간 의존성이 빠르게 커질 수 있다는 점이 가장 큰 과제였습니다. 그래서 도메인 규칙, 애플리케이션 포트, 외부 연동 어댑터, 조립 책임을 나누고, 요청이 게이트웨이에서 실행·추적으로 이어지는 경로와 리포팅 서비스가 근거를 다루는 경계를 구분해 완성했습니다. 이후 패키지별 타입체크와 테스트, 저장소에 정의된 health/readiness 및 결정적 smoke 경로를 기준으로 통합 조건을 점검했습니다. 이 과정에서 완수란 코드를 많이 작성하는 것이 아니라, 기능의 시작과 끝을 설명하고 변경·실패를 점검할 수 있게 만드는 일임을 배웠습니다.

평가 포인트: 프로젝트 종류·규모·역할·수행 과정·학습 · 직접 연결되지 않은 Reporting 경계는 분리 표현

3. 삶의 가치관과 회사에서의 실현 방법

392자

저의 가치관은 '불편함을 발견하면 개선하려 노력하는 것'입니다. 저는 불편을 개인의 인내심 문제로 보지 않고, 반복 비용이나 책임 경계가 드러내는 시스템의 신호로 봅니다. 따라서 곧바로 도구를 추가하기보다 누가 언제 어떤 흐름에서 막히는지 확인하고, 원인을 작게 쪼갠 뒤 개선 효과를 검증하려 합니다. 회사에서도 현업의 반복 입력, 개발·운영 사이의 확인 누락, 장애 원인을 찾기 어려운 절차를 발견하면 관련 구성원과 문제를 같은 언어로 정의하겠습니다. 그 다음 자동화, 명시적인 계약, 관찰 가능한 로그나 점검 절차 중 가장 작은 해결책을 제안하고 실제 업무에 적용되는지 확인하겠습니다. 이렇게 불편을 구조적 개선으로 바꾸고, 개인의 편의를 넘어 팀의 생산성과 AI 서비스 품질을 함께 높이겠습니다.

평가 포인트: 가치관을 관찰·정의·검증·협업 행동으로 번역

세트 2 · 경계를 설계해 복잡성을 낮추는 엔지니어

모듈 경계·책임 분리·팀 자산화에 무게를 둔 안

문항 1	문항 2	문항 3	세트
93	92	90	92 · PASS

1. 지식 및 기술역량을 지원직무에서 발휘하는 방법

445자

AI 서비스는 모델 성능만으로 완성되지 않고, 모델 앞뒤의 인증·입력 변환·상태 관리·결과 해석·품질 확인이 함께 안정적으로 작동해야 사용자 가치가 생깁니다. 저는 서비스의 복잡성을 한곳에 몰아넣기보다 책임을 경계로 나누고, 그 경계를 통해 변화가 번지는 범위를 통제하는 역량을 갖추고 있습니다. TypeScript 기반 프로젝트에서 도메인 규칙은 순수하게 유지하고, 애플리케이션은 필요한 계약을 정의하며, 전송·저장·외부 SDK는 어댑터와 조립 계층에서 연결했습니다. 그 결과 통신 방식이 달라져도 핵심 업무 흐름을 다시 작성하지 않고 검증할 수 있었습니다. 지원 직무에서도 AI 요청의 입력부터 결과와 예외 처리까지를 하나의 서비스 흐름으로 바라보겠습니다. 기능을 빠르게 붙이는 데 그치지 않고, 변경 가능한 부분과 반드시 지켜야 할 계약을 구분해 안정적인 API와 운영 기준을 만들겠습니다.

평가 포인트: 기술을 책임 경계와 변경 영향 범위의 언어로 설명

2. 프로젝트를 완수하기 위해 노력한 경험

481자

collab-mono는 Markdown과 .collab.yaml을 해석해 문서의 의미를 AST와 그래프로 투영하고, 이를 에이전트 실행·협업·리포팅으로 연결하는 개인 프로젝트입니다. 여러 도메인 패키지와 웹·MCP·인프라 실행 경로를 포함해, 저는 요구 분석부터 모듈 책임 정의, 코드 구현, 통합 검증까지 전 과정을 맡았습니다. 초기에 문서 해석과 실행 로직을 한곳에서 처리하면 작은 변경도 전체 흐름에 영향을 줄 수 있었습니다. 그래서 문서의 의미를 표현하는 영역과 실제 실행·전송을 담당하는 영역을 나누고, 애플리케이션 포트를 사이에 두어 각 모듈이 협력하는 방식을 명확히 했습니다. 이후 게이트웨이와 런타임의 요청 흐름, 리포팅 서비스의 별도 책임 경계를 각각 확인하며 경계 밖의 의존성을 줄였습니다. 이 경험을 통해 좋은 구조는 추상적인 규칙이 아니라, 새로운 요구를 안전하게 받아들이고 문제의 위치를 빠르게 찾게 하는 협업 언어라는 점을 배웠습니다.

평가 포인트: 문서 의미·실행·전송을 분리한 이유와 협업 가치

3. 삶의 가치관과 회사에서의 실현 방법

391자

저는 개선을 '더 많은 도구를 도입하는 일'보다 '반복되는 판단을 명확한 흐름으로 바꾸는 일'에 가깝게 생각합니다. 업무에서 같은 질문이 반복되거나 담당자마다 처리 방식이 달라진다면, 먼저 그 판단에 필요한 정보와 책임 주체를 정리합니다. 그 뒤 체크리스트, 자동화, API 계약, 알림 중 실제 병목을 줄이는 방법을 선택하고, 사용자가 다시 우회 절차를 만들지 않는지 확인합니다. 회사에서는 개발팀의 기술적 편의만을 기준으로 삼지 않고 현업의 업무 맥락과 운영팀의 대응 부담까지 함께 살피겠습니다. 이렇게 서로 다른 관점을 하나의 흐름으로 연결하면 개선안이 특정 사람의 노하우에 머무르지 않고 팀의 자산이 된다고 믿습니다. 저의 가치관을 협업 방식으로 실천해 조직의 변화가 지속되도록 만들겠습니다.

평가 포인트: 개인의 개선 의지를 조직의 재사용 가능한 기준으로 확장

세트 3 · 운영 가능한 결과와 검증 가능한 근거

품질 기준·실패 위치·재현 가능성에 무게를 둔 안

문항 1	문항 2	문항 3	세트
92	91	90	91 · PASS

1. 지식 및 기술역량을 지원직무에서 발휘하는 방법

467자

제가 가진 지식과 기술역량은 기능을 구현하는 능력에 운영 가능성의 관점을 더하는 것입니다. 서비스가 실제로 사용되려면 정상 경로뿐 아니라 실패가 어디에서 발생했는지 확인하고, 수정 후 같은 문제가 재발하지 않았는지 검증할 수 있어야 합니다. 저는 TypeScript와 모듈 경계 설계를 바탕으로 업무 규칙과 외부 연동을 분리하고, 타입체크·테스트·health/readiness·smoke 검증을 서로 다른 안전망으로 활용해 왔습니다. 각각은 단순한 도구가 아니라 변경 전 구조를 확인하고, 실행 환경의 준비 상태를 구분하며, 핵심 경로의 연결 조건을 점검하는 장치입니다. AI 서비스 엔지니어로서도 API와 데이터 흐름을 설계할 때 품질 기준과 관찰 지점을 함께 정의하겠습니다. 모델이나 외부 서비스의 응답이 달라져도 영향 범위를 파악하고 복구 가능한 운영 흐름을 만들어, 신기술을 실제 업무에서 지속적으로 사용할 수 있게 하겠습니다.

평가 포인트: 체크 도구를 운영 안전망의 역할로 설명 · 과거 실행 성과와 구조상 검증 기준을 구분

2. 프로젝트를 완수하기 위해 노력한 경험

496자

collab-mono는 여러 도메인 컴포넌트와 웹·MCP·인프라·E2E 영역을 포함한 TypeScript 모노레포 개인 프로젝트입니다. 저는 혼자 요구를 정리하고 핵심 서비스 경로를 설계·구현한 뒤, 개발 환경에서 동작하는 것과 다른 구성요소가 함께 준비된 것을 구분해 점검했습니다. 프로젝트를 진행하며 가장 어려웠던 점은 한 패키지의 성공이 전체 서비스의 완성을 의미하지 않는다는 사실이었습니다. 이를 해결하기 위해 핵심 요청이 게이트웨이에서 실행 모듈을 거쳐 결과·추적 정보로 이어지는 경로와, 별도로 리포팅 서비스가 근거를 다루는 경계를 나누고, 패키지별 타입체크와 테스트, 컨테이너·서비스 readiness, 결정적 smoke 검증을 단계별 기준으로 적용했습니다. 오류가 생기면 코드를 바로 수정하기보다 입력·경계·실행 상태 중 어느 층에서 문제가 생겼는지 좁혀 갔습니다. 그 결과 완수의 기준을 '구현 완료'에서 '재현하고 설명할 수 있는 동작'으로 바꾸어 생각하게 되었습니다.

평가 포인트: 구현 성공과 전체 준비 상태를 구분 · 문제 위치를 단계적으로 좁힌 과정

3. 삶의 가치관과 회사에서의 실현 방법

409자

저의 가치관은 불편함을 발견했을 때 원인을 확인하고 개선으로 연결하는 것입니다. 다만 모든 불편을 즉시 바꾸는 것이 좋은 개선은 아니라고 생각합니다. 반복 횟수, 영향받는 사람, 실패 비용을 살펴 우선순위를 정하고, 바꾼 뒤 무엇이 나아졌는지 확인할 수 있는 기준을 세워야 합니다. 회사에서도 AI 서비스의 응답 지연, 반복 수작업, 확인되지 않은 예외처럼 작아 보이지만 누적되는 불편을 발견하면 현상을 먼저 기록하겠습니다. 이후 원인을 업무 절차와 시스템 동작으로 나누어 보고, 작은 자동화나 점검 기준으로 실험한 뒤 효과를 공유하겠습니다. 이런 방식은 개선을 개인의 감각이 아닌 팀이 재사용할 수 있는 학습으로 바꿉니다. 저는 불편을 숨기지 않고 품질 신호로 활용해, 서비스와 업무 방식이 함께 좋아지는 문화를 만들고 싶습니다.

평가 포인트: 불편을 관찰 가능한 품질 신호와 우선순위로 변환

세트 4 · 불확실한 요구를 검증 가능한 계약으로

외부 AI 기술 통합·계약·대안 비교에 무게를 둔 안

문항 1	문항 2	문항 3	세트
94	93	92	93 · PASS

1. 지식 및 기술역량을 지원직무에서 발휘하는 방법

466자

저는 모호한 요구를 그대로 코드로 옮기기보다, 사용자가 기대하는 결과와 시스템이 책임져야 할 경계를 먼저 정의하는 역량을 갖추고 있습니다. 특히 AI 서비스는 입력의 형태와 품질, 외부 모델·도구의 응답, 상태 변화가 함께 결과를 만들기 때문에 '호출 성공'만으로는 품질을 판단하기 어렵습니다. 저는 서비스 흐름을 작은 계약으로 나누고, 각 단계의 입력·출력·실패 조건을 확인하는 방식으로 설계해 왔습니다. TypeScript 모듈에서는 도메인 규칙과 실행 절차를 분리하고, API·WebSocket 같은 사용자 접점과 MCP 같은 외부 도구 접점을 어댑터로 격리해 핵심 흐름이 특정 통신 방식에 묶이지 않도록 했습니다. 입사 후에는 현업 요구를 기술 용어로 치환하는 데 그치지 않고, 함께 확인할 수 있는 시나리오와 품질 기준으로 번역하겠습니다. 이를 통해 빠른 실험과 안전한 운영이 서로 충돌하지 않는 AI 서비스를 만들겠습니다.

평가 포인트: MCP와 실행 표면을 구분 · 불확실한 요구를 입력·출력·실패 계약으로 전환

2. 프로젝트를 완수하기 위해 노력한 경험

456자

collab-mono는 에이전트 기반 협업 기능을 중심으로 대화, 실행, 리포팅, 웹, MCP, 인프라를 연결하는 개인 프로젝트입니다. 저는 전체 구조와 각 모듈의 책임을 정하고, 요구가 바뀌어도 핵심 규칙이 흔들리지 않도록 대안을 비교하며 구현했습니다. 처음부터 모든 기능을 완성하려 하지 않고, 사용자의 요청이 들어와 실행되고 결과가 추적되는 최소 경로를 먼저 만들었습니다. 그 과정에서 전송 계층의 편의를 도메인에 직접 반영하면 새로운 채널을 추가할 때 변경 범위가 커진다는 문제를 확인했습니다. 그래서 포트와 컴포지션 계층을 두고, 외부 시스템과의 차이를 어댑터에서 흡수했습니다. 각 단계마다 타입체크·테스트·readiness 검증을 실행해 설계 의도가 실제 코드와 맞는지도 확인했습니다. 이 경험은 문제를 빨리 해결하는 능력뿐 아니라, 대안의 비용과 미래 변경까지 설명하며 선택하는 태도를 길러 주었습니다.

평가 포인트: 최소 경로 우선 완성 · 전송 편의와 도메인 책임을 분리한 판단

3. 삶의 가치관과 회사에서의 실현 방법

394자

제가 중요하게 여기는 가치는 불편함을 발견하면 더 나은 상태를 만들기 위해 움직이는 것입니다. 조직의 불편은 한 사람의 실수보다 서로 다른 팀이 같은 문제를 다르게 이해할 때 커지는 경우가 많습니다. 그래서 저는 개선을 제안하기 전에 현재 흐름, 책임 주체, 사용되는 용어를 함께 확인하고, 각자가 동의할 수 있는 문제 문장부터 만들려고 합니다. 회사에서는 개발·기획·운영의 요구를 연결하는 과정에서 반복되는 확인이나 해석 차이를 발견하면, 예시 시나리오와 명시적인 계약으로 합의 내용을 남기겠습니다. 이후 작은 범위에 적용해 실제 업무 부담과 서비스 품질이 줄었는지 확인하고 확장하겠습니다. 이렇게 개선을 개인의 주장으로 끝내지 않고 공동의 기준으로 정착시키는 방식으로 협업과 성과를 함께 만들겠습니다.

평가 포인트: 불편을 공통 언어·시나리오·계약으로 바꾸는 협업 방식

세트 5 · 기술을 고객과 조직의 실제 가치로

고객 업무·사용자 불편·지속 가능한 서비스 경쟁력에 무게를 둔 안

문항 1	문항 2	문항 3	세트
91	90	90	91 · PASS

1. 지식 및 기술역량을 지원직무에서 발휘하는 방법

471자

제가 발휘할 수 있는 역량은 새로운 기술을 빠르게 붙이는 데서 멈추지 않고, 그것이 사용자의 업무와 조직의 운영에 어떤 변화를 만드는지까지 설계하는 것입니다. AI 서비스는 기술 시연보다 사용자가 결과를 믿고 다음 업무를 이어 갈 수 있을 때 가치가 생깁니다. 저는 서비스의 요구를 도메인 흐름으로 정리하고, TypeScript 기반 모듈 구조와 외부 연동 경계를 통해 변경 가능성을 관리해 왔습니다. collab-mono에서 대화·게이트웨이·에이전트 실행과 리포팅 서비스를 각각의 책임으로 구분한 경험은 요청을 전달하는 것뿐 아니라 실행 이력과 결과를 어떤 경계에서 확인할지 정해야 서비스가 설명 가능해진다는 점을 보여 주었습니다. 입사 후에는 AI 기능을 현업의 업무 단계에 맞춰 설계하고, 사용자 피드백과 운영 기록을 다음 개선의 근거로 삼겠습니다. 기술과 비즈니스 사이의 간극을 줄여 실제로 쓰이고 계속 좋아지는 서비스를 만들겠습니다.

평가 포인트: 기술을 고객 업무의 다음 단계와 설명 가능성으로 연결

2. 프로젝트를 완수하기 위해 노력한 경험

479자

collab-mono는 AI·에이전트 협업 흐름을 서비스 형태로 실험하는 개인 프로젝트로, 도메인 모듈뿐 아니라 웹·MCP·인프라·E2E 경로까지 포함한 모노레포입니다. 저는 아이디어를 기능 목록으로 쌓기보다 사용자의 요청이 들어와 실행되고, 결과를 확인할 수 있는 여정을 먼저 설계했습니다. 혼자 진행했기 때문에 기획, 설계, 구현, 테스트, 배포 구성을 모두 스스로 연결해야 했고, 범위가 커질수록 우선순위를 지키는 일이 중요했습니다. 핵심 요청·실행·trace 경로와 별도 리포팅 서비스의 책임을 작은 단위로 정리한 뒤 모듈별 검증과 서비스 readiness를 점검하고, 그 다음 주변 기능을 확장했습니다. 또한 업무 규칙과 외부 시스템 연결을 분리해 한 기능의 변경이 전체 구조를 흔들지 않게 했습니다. 이 경험을 통해 프로젝트의 완수는 기능 수가 아니라 사용자 흐름의 끝까지 책임지고, 다음 개선이 가능한 근거를 남기는 데 있다는 것을 배웠습니다.

평가 포인트: 범위가 넓은 개인 프로젝트에서 핵심 여정 우선·점진적 확장

3. 삶의 가치관과 회사에서의 실현 방법

410자

저의 가치관은 불편함을 발견하면 그것을 당연한 비용으로 넘기지 않고 개선의 출발점으로 삼는 것입니다. 특히 사용자가 여러 화면을 오가거나 같은 정보를 반복 입력해야 하는 문제는, 서비스가 기술적으로 동작하더라도 고객 경험을 떨어뜨립니다. 회사에서는 먼저 사용자의 업무 목적과 현재 우회 방법을 이해한 뒤, 꼭 필요한 정보와 단계만 남기는 방향으로 개선을 제안하겠습니다. 자동화가 답이라면 예외 상황과 운영 책임까지 포함하고, 구조 변경이 필요하다면 영향을 받는 팀과 작은 실험으로 검증하겠습니다. 결과는 편리해졌다는 느낌에만 의존하지 않고 처리 단계, 문의, 오류와 같은 관찰 가능한 신호로 확인하겠습니다. 이렇게 기술을 고객의 시간과 조직의 신뢰를 아끼는 방식으로 연결하며, 작은 개선을 지속 가능한 서비스 경쟁력으로 키우겠습니다.

평가 포인트: 불편을 고객 시간·신뢰·관찰 가능한 신호로 번역

5. 점수의 의미와 제출 전 게이트

ALL FIVE INCLUDED

세트	핵심 맥락	Q1/Q2/Q3	종합	점수의 의미
1	전체 흐름·운영 가능한 AI 서비스	94 / 95 / 93	**94**	가장 균형적인 기본안; 직무·프로젝트·운영 연결이 선명
2	경계·복잡성·팀 자산화	93 / 92 / 90	**92**	모듈 경계가 강점; 개인 변경 사례를 추가하면 더 방어적
3	검증·실패 위치·운영 근거	92 / 91 / 90	**91**	품질 언어가 선명; 실제 실행 결과는 별도 확인 필요
4	외부 AI 통합·계약·대안	94 / 93 / 92	**93**	메커니즘 정확도가 높고 MCP/실행 표면을 구분
5	고객 업무·불편·서비스 가치	91 / 90 / 90	**91**	사용자 관점이 강점; 실제 고객/운영 사례 확인 권장

92.2 / 100 · 평균

모든 세트가 90점 이상이며, 각 세트의 문항도 90점 이상으로 평가했습니다. 90점 미만 후보는 결과물에서 제외하는 규칙을 적용했습니다.

추천 순서

1순위 세트 1 — 공고가 요구하는 서비스 엔지니어 관점과 현재 프로젝트 근거의 균형이 가장 좋습니다. **2순위 세트 4** — AI 기술 통합과 기술 면접 방어를 강조할 때 적합합니다.

면접 방어 질문

어떤 모듈/파일을 바꿨는가? provider·auth·readiness가 실패하면 어떻게 되는가? Chat Gateway와 Reporting의 경계는 무엇인가? 무엇을 실제로 측정했고 무엇은 아직 측정하지 않았는가?

제출 직전 확인

- 본인의 실제 모듈·변경·기여 범위 확인
- 실행한 테스트·배포·검증 결과만 과거형으로 확정
- 각 기술 문단의 문제·메커니즘·품질 신호 확인
- 고객·성과·수치가 없으면 미래 계획으로 유지
- 학교·학과·수업 등 학력 유추 정보 제거
- 개인 경험과 맞지 않는 문장은 해당 세트에서 제외

부속 산출물: evaluation-rubric-v2.json · evidence-map-v2.json · quality-loop-v2.json · self-introduction-content-v2.json